

LLMsVerifier

LLMsVerifier

Vérifier. Surveiller. Optimiser.

Résumé

LLMsVerifier est une plateforme de niveau entreprise conçue pour vérifier, surveiller et optimiser les grands modèles de langage (LLM) auprès de multiples fournisseurs. Elle repose sur un test de vérification obligatoire « *Voyez-vous mon code ?* », garantissant que seuls les modèles ayant prouvé leur fonctionnement effectif sont marqués comme utilisables ou exportés.

Description courte

Une plateforme Go qui vérifie, évalue, surveille et optimise les LLM auprès de plusieurs fournisseurs. Chaque modèle doit réussir un test obligatoire de visibilité du code avant utilisation ; il subit ensuite des contrôles de latence, de diffusion en continu, d'appel de fonctions, de vision et d'intégration, puis exporte des configurations vérifiées exclusivement pour les outils AI et CLI.

Description longue

LLMsVerifier est une plateforme complète dédiée à la vérification, à la surveillance et à l'optimisation des performances des LLM auprès de différents fournisseurs. Son principe fondamental est la *vérification obligatoire*, et elle ne transige pas : avant qu'un modèle ne soit marqué comme utilisable – ou autorisé dans une configuration exportée – il doit impérativement réussir un test « *Voyez-vous mon code ?* ». Ce test effectue des appels HTTP réels vers le fournisseur et analyse la réponse pour s'assurer d'une compréhension authentique, et non d'un simple écho plausible. Un modèle incapable de démontrer qu'il voit et comprend votre entrée n'obtiendra jamais le statut « utilisable ». Une fois cette étape validée, le moteur de vérification exécute une batterie

complète de tests de capacités – existence, réactivité, latence, diffusion en continu, appel de fonctions, vision, embeddings – tandis que le moteur de rapport transforme les résultats en rapports markdown et JSON exploitables.

Le système est modulaire et piloté par événements, proposant des interfaces CLI, TUI, Web et REST/API reposant sur un cœur composé du moteur de vérification, du moteur de rapport et du gestionnaire de configuration. Mais il ne s'arrête pas à la vérification. Des couches avancées ajoutent un modèle Superviseur/Worker pour la décomposition des tâches alimentées par LLM, une gestion du contexte par fenêtre glissante avec résumé LLM pour éviter que les sessions très longues ne s'effondrent, un système de points de contrôle sauvegardés dans le cloud, ainsi qu'un mécanisme de bascule intégrant des disjoncteurs et un routage basé sur la latence. L'infrastructure environnante est conçue pour la production : un bus d'événements pub/sub, une planification cron, une détection des tarifs et limites, une base de données vector pour les RAG, et un système d'export. Une convention de marquage signature ajoute le suffixe (*llmsvd*) à chaque fournisseur/modèle généré, permettant d'identifier d'un coup d'œil une sortie vérifiée et d'éviter toute confusion avec une version non validée. Seuls les modèles vérifiés sont intégrés aux configurations exportées pour les outils AI et CLI comme OpenCode, Crush et Claude Code. La plateforme est livrée avec les outils opérationnels dont les équipes ont réellement besoin en production : déploiement Docker/Kubernetes/Helm, surveillance Prometheus/Grafana, authentification LDAP/SSO et stockage chiffré SQLCipher.

Pourquoi nous l'avons créée

Parce que la vérification par simple configuration est peu fiable : une clé API peut expirer, un modèle peut être obsolète, et un fichier de configuration ne vous dit rien sur la latence réelle, les erreurs effectives, ou si le modèle est réellement capable de voir et de comprendre votre entrée. LLMsVerifier remplace « *c'est dans la config, donc ça doit marcher* » par des preuves : seuls les modèles ayant démontré une réponse correcte sont marqués comme utilisables et exportés.

Contenu

Pourquoi c'est un changement de paradigme

Il rend les flottes LLM *fiables* — un qualificatif rarement mérité dans un espace où les configurations mentent par omission. Au lieu d'espérer qu'un modèle configuré fonctionne, les équipes

obtiennent une garantie imposée et vérifiable : chaque modèle en service a passé une validation réelle, avec un suivi, un basculement automatique et une exportation réservée aux modèles vérifiés, bouclant ainsi le cycle de la preuve à la production. Au sein de l'écosystème Helix, il devient la source unique de vérité pour les métadonnées des modèles, fournisseurs et vérifications LLM : d'autres services (dont HelixTranslate) s'y réfèrent, si bien que toute la plateforme hérite d'une réponse honnête à la question « *Quels modèles fonctionnent réellement en ce moment ?* », au lieu que chaque équipe maintienne sa propre estimation optimiste.

Ce qui est innovant

- **Vérification obligatoire « *Vois-tu mon code ?* »** — une porte de compréhension réelle, adossée à HTTP, qu'un modèle doit franchir avant d'être utilisable ; la signature distinctive du produit et la raison pour laquelle rien de non validé ne passe entre les mailles du filet.
- **Exportation de configurations réservées aux modèles vérifiés** — les fichiers de configuration générés pour les outils AI CLI ne contiennent *que* des modèles ayant passé la vérification, empêchant ainsi qu'une configuration déployée ne réintroduise discrètement un modèle défectueux.
- **Système de suffixe de marque (llmsvd)** — chaque fournisseur ou modèle généré porte un suffixe traçable, rendant la provenance vérifiée visible partout où les résultats circulent.
- **Détection des capacités** sur de nombreux agents et fournisseurs CLI — il identifie les types de streaming (SSE, WebSocket, JSONL, EventStream), la compression et les comportements de mise en cache, plutôt que de les supposer.
- **Basculement résilient** — des disjoncteurs, un routage basé sur la latence qui redirige le trafic lorsque le temps jusqu'au premier jeton dépasse un seuil, des sondes de santé et une répartition pondérée du trafic maintiennent la réactivité d'une flotte même lorsque certains fournisseurs vacillent.
- **Autonomie sur de longues sessions** — un modèle de décomposition Superviseur/Travailleur, associé à des points de contrôle périodiques et à une intégration de la mémoire, permet de soutenir des sessions prolongées qui, sinon, épuiseraient le contexte.
- **Intégration RAG / vector-DB** pour un enrichissement contextuel ancré.

Principaux défis techniques et nos solutions

- **Prouver qu'un modèle fonctionne réellement, et pas seulement qu'il est configuré.** L'objectif même, et la partie la plus ardue. Résolu par le test obligatoire de visibilité du code, qui effectue de véritables appels API et analyse les réponses pour confirmer la compréhension, soutenu par une large batterie de tests de capacités — puis en refusant d'exporter tout ce qui n'a pas passé ces tests, de sorte que ce soit la preuve, et non la configuration, qui conditionne la mise en production.
- **Fiabilité face à de nombreux fournisseurs tiers peu fiables.** Résolu par un orchestrateur de basculement qui traite l'instabilité des fournisseurs comme un cas normal : les disjoncteurs marquent un fournisseur comme dégradé après N échecs en M secondes, le routage basé sur la latence évite les points d'accès lents, des vérifications de santé périodiques sondent la reprise, et un routage pondéré équilibre les modèles économiques et premium.
- **Maintien de sessions autonomes très longues.** Résolu par le modèle de décomposition Superviseur/Travailleur, qui découpe les gros travaux en tâches gérables, des points de contrôle périodiques vers un stockage cloud pour préserver l'avancement en cas d'interruption, et une gestion contextuelle en couches (fenêtre glissante + résumé LLM + RAG) afin que le modèle conserve le fil sans se noyer dans les jetons.
- **Prolifération des fournisseurs.** Résolu en masquant de nombreux adaptateurs Go spécifiques à chaque fournisseur derrière une interface commune, avec une énumération centralisée des points d'accès réels — ainsi, ajouter un fournisseur devient un changement localisé, sans répercussions en cascade dans le code.

Contenu

Pile technologique

- **Go** — choisi comme langage de base pour sa gestion de la concurrence ; il anime un moteur de vérification multithread capable de sonder plusieurs modèles en parallèle, ainsi que les services environnants.
- **Gin** — retenu comme serveur REST API, prenant en charge l'authentification JWT, la limitation de débit et les points de terminaison WebSocket/SSE.
- **SQLite + SQLCipher** — sélectionnés pour le stockage embarqué avec chiffrement au niveau de la base de données, car les données de vérification (clés, résultats) sont sensibles et doivent être chiffrées par défaut au repos.

- **Redis** — adopté comme couche de mise en cache pour accélérer les recherches de vérifications et de métadonnées fréquentes.
- **RabbitMQ + Kafka** — utilisés pour alimenter l'architecture pilotée par événements : messagerie et streaming qui dissocient les producteurs des consommateurs au sein de la plateforme.
- **gRPC + Protocol Buffers** — choisis pour une communication inter-services et un transport d'événements fortement typés entre les composants.
- **QUIC / HTTP-3 (quic-go)** — retenus pour leur support des protocoles de transport modernes (la documentation du dépôt indique que la disponibilité des fournisseurs HTTP/3 est limitée — une capacité proposée, non une affirmation universelle).
- **JWT + LDAP/NTLM** — adoptés pour l'authentification d'entreprise, permettant à la plateforme de s'intégrer aux systèmes d'identité existants (SSO/SAML/OIDC mentionnés dans la documentation).
- **Viper (configuration), Logrus (journalisation), Brotli/compress (compression)** — les rouages opérationnels : configuration flexible, journaux structurés et compression des charges utiles.
- **Angular** — choisi pour l'application web monopage, la vitrine visuelle de la vérification et de la supervision.
- **Python + SDK JavaScript** — retenus pour offrir aux équipes clientes un accès de premier ordre, documenté via OpenAPI/Swagger.
- **Docker, Kubernetes, Helm** — sélectionnés pour le déploiement en production avec surveillance de l'état et autoscaling, permettant à une flotte de vérification de s'adapter comme tout service moderne.
- **Prometheus + Grafana** — adoptés pour les métriques et tableaux de bord, rendant la santé de la plateforme aussi observable que celle des modèles qu'elle surveille.
- **Testify (Go) + node -test/jsdom (web)** — utilisés pour des tests en couches couvrant le cœur Go et l'interface web.

Statut et notes de transparence

- **Statut : bêta.** Le code source en Go implémente une vérification HTTP réelle (un document obsolète décrivant la vérification comme uniquement basée sur la configuration relève de l'aspiration et n'est plus à jour — le code fait autorité).
- **Licence : à déterminer.** Le fichier README indique une licence MIT, tandis qu'une étiquette de Dockerfile mentionne Apache-2.0 — à résoudre avant publication.
- **Nombre de fournisseurs :** le README annonce « 12 adaptateurs », mais le répertoire *providers* en recense environ

26 — à considérer comme « 12+ / en cours d'ajout ». De nombreux fichiers de statut « FINAL/COMPLET » ambitieux existent ; le code, la documentation et le fichier `go.mod` font foi.

- Le dépôt est hébergé dans l'organisation `vasic-digital`, mais il constitue fonctionnellement la couche de confiance du cluster d'infrastructure Helix LLM.

Niveau de priorité : Helix-primaire (cluster d'infrastructure LLM ; source unique de vérité pour les métadonnées de LLM/fournisseur/vérification). Se classe après HelixTrack.